AutoSRE: cloud handoff report

Live validation on AWS and what the cloud role needs to do. Repo: vaishnavi-nss/autosre-devops. Status as of 13 Nov 2026.

Where the code is

- **ml-cloud-additions** (06e52d4): the PR branch. Adds files only; nothing on main is modified.
- **ml-and-cloud-validation** (272c185): full history, including files that overlap with main.
- The pull request is not opened; its base branch must be main.

Live validation

AWS Academy lab, one t3.large (unlimited CPU credits) with minikube and kube-prometheus-stack. Faults were injected through the CHAOS_ENABLED endpoints; the alert rules, Alertmanager, webhook and engine were the real pipeline. Sessions run 1 to 15 (run 10 was void: a deployment mistake, no data).

Measure	Result
Rule engine, runs 1-2	32 injected-fault trials, 0 wrong actions
Time for an incident to reach the engine	about 2 minutes after injection
Alerts clear after a fault ends	about 5 minutes (rate()[5m] windows)
Second-opinion scorer, runs 11-14 (retrained model)	46 judged decisions, 0 vetoes, upper bound 7.7%; gate bar met; every miss a no_opinion
Trials lost to contamination, runs 11-14	15 of 46

The finding that matters for the cloud role

The services' 200m CPU limits are too low for the traffic. They throttle about 22% of CPU periods while the node is only about 45% busy, which puts the idle gateway p95 (0.45-0.77 s) on the 0.5 s line of APILatencyHigh. So that alert can fire with no fault present and then will not clear, and every service is slower than it needs to be. Measured on the live cluster: at 1000m the idle p95 was about 0.2 s (transaction-service 0.07 s, was 0.25-0.44 s) and the alert stayed quiet.

- Raise the CPU limits in k8s/api-gateway.yaml, transaction-service.yaml, database.yaml and frontend.yaml. These files exist on main, so the PR does not touch them. The memory limit (128Mi) is what the memory fault depends on; leave it.
- If the limits must stay low, raise the 0.5 s threshold in monitoring/alert-rules.yaml, or lengthen its for: clause; the healthy range is inside it.
- Until then, the harness takes *--cpu-limit 1000m* and logs it.

What to do by hand

These files already exist on main, so the PR leaves them out: scripts/build-images.sh (also build autosre/second-opinion:dev), scripts/deploy.sh (wait for the second-opinion rollout), k8s/correlation-engine.yaml (set SECOND_OPINION_URL and SECOND_OPINION_MODE=shadow). k8s/second-opinion.yaml is new and included. Set the dispatcher cooldown to at least 10 minutes: alerts keep firing about 5 minutes after a fault ends.

Running a session

Press Start Lab, wait for green, open the console with the lab's own AWS link (a restart signs the console out), then in CloudShell: *curl -fsS https://raw.githubusercontent.com/vaishnavi-nss/autosre-devops/ml-and-cloud-validation/scripts/aws/start-live-run.sh | RUN=run16 PER_CAUSE=3 UNKNOWN=0 DEADLINE=120 bash*. It rebuilds the images, replaces the pods, runs the harness and syncs results to the bucket every 45 s. Allow about 12 minutes to prepare and 6-14 minutes per trial; the lab gives 4 hours. The harness stops starting trials at the deadline.

AWS Academy limits

- Instance types above large are denied. No IAM role creation: use LabInstanceProfile. Session about 4 hours.
- CloudShell drops idle sessions; reconnect and retype. The S3 console serves .jsonl files as downloads.

Still open

- Bucket autosre-validation-841736140404 (raw results of the sessions), the instance i-04f8f4d75378a1aba and an unused security group are kept on purpose; nothing was cleaned up. The VLE*-jenkins instances in the lab account may still be running; they were not touched.
- Recommendation unchanged: keep SECOND_OPINION_MODE=shadow. The scorer now meets the written gate bar on our own data, so enabling it is a decision for the team, one cause at a time.